

SETTER IN DART

Setter In Dart

Setter is used to set the value of a property. It is mostly used to update a **private property's** value. Setter provide explicit write access to an object properties.

Syntax

```
set property_name (value) {  
    // Setter body  
}
```



Info

Note: Instead of writing { } after the property name, you can also write **=>** (fat arrow) after the property name.

Example 1: Setter In Dart

In this example below, there is a class named **NoteBook**. The class has two private properties **_name** and **_prize**. There are two setters **name** and **price** to update the value of the properties. There is also a method **display** to display the value of the properties.

```
class NoteBook {  
    // Private Properties  
    String? _name;  
    double? _prize;  
  
    // Setter to update private property _name  
    set name(String name) => this._name = name;  
  
    // Setter to update private property _prize  
    set price(double price) => this._prize = price;  
  
    // Method to display the values of the properties  
    void display() {
```



```

        print("Name: ${_name}");
        print("Price: ${_prize}");
    }
}

void main() {
    // Create an object of Notebook class
    Notebook nb = new Notebook();
    // setting values to the object using setter
    nb.name = "Dell";
    nb.price = 500.00;
    // Display the values of the object
    nb.display();
}

```

[Show Output](#)

Run Online

Info

Note: In the above example, a setter **name** and **price** are used to update the value of the properties **_name** and **_prize**.

Example 2: Setter In Dart With Data Validation

In this example, there is a class named **NoteBook**. The class has two private properties **_name** and **_prize**. If the value of **_prize** is less than 0, we will throw an exception. There are also two setters **name** and **price** to update the value of the properties. The class also has a method **display()** to display the values of the properties.

```

class Notebook {
    // Private properties
    String? _name;
    double? _prize;

    // Setter to update the value of name property
    set name(String name) => _name = name;

    // Setter to update the value of price property
    set price(double price) {
        if (price < 0) {
            throw Exception("Price cannot be less than 0");
        }
        this._prize = price;
    }
}

```

```

}

// Method to display the values of the properties
void display() {
    print("Name: $_name");
    print("Price: $_prize");
}

}

void main() {
    // Create an object of NoteBook class
    Notebook nb = new Notebook();
    // setting values to the object using setter
    nb.name = "Dell";
    nb.price = 250;

    // Display the values of the object
    nb.display();
}

```

[Show Output](#)

Run Online

Info

Note: It is generally best to not allow the user to set the value of a field directly. Instead, you should provide a setter method that can validate the value before setting it. This is very important when working on large and complex programs.

Example 3: Setter In Dart

In this example, there is a class named **Student**. The class has two private properties **_name** and **_classnumber**. We will also create two setters **name** and **classnumber** to update the value of the properties. The **classnumber** setter will only accept a value between 1 and 12. The class also has a method **display()** to display the values of the properties.

```

class Student {
    // Private properties
    String? _name;
    int? _classnumber;

    // Setter to update the value of name property
    set name(String name) => this._name = name;
}

```



```
// Setter to update the value of classnumber property
set classnumber(int classnumber) {
    if (classnumber <= 0 || classnumber > 12) {
        throw ('Classnumber must be between 1 and 12');
    }
    this._classnumber = classnumber;
}

// Method to display the values of the properties
void display() {
    print("Name: $_name");
    print("Class Number: $_classnumber");
}
}

void main() {
    // Create an object of Student class
    Student s = new Student();
    // setting values to the object using setter
    s.name = "John Doe";
    s.classnumber = 12;

    // Display the values of the object
    s.display();

    // This will generate error
    //s.setClassNumber(13);
}
```

[Show Output](#)

Run Online

Why Is Setter Important?

- It is used to set the value of a private property.
- It is also used for data validation.
- It gives you better control over the data.

Video

Watch our video on setters in Dart.

Challenge

Try to create a class named **University** with two private properties **_name** and **_year**. The class will also have two setters **name** and **year** to update the value of the properties. The

year setter will only accept a value between 1900 and 2023. Also, create a method **display()** to display the values of the properties.